

Timsort

Group 5

Contents

Introduction	2
Methodology	3
Empirical Run-Time Testing Methodology	3
Data generation	3
Computer Specifications	3
Data Collection Methods	6
Time complexity analysis	7
Performance results	8
Runtime	8
Conclusion	9
• [] Algorithm explanation with examples	
• [] Time complexity analysis	
• [] Java implementation	
☒ Testing methodology	
• [] Performance results for 5, 1,000, and 1,000,000 elements	
• [] Comparison with Bubble Sort, Selection Sort, and Insertion Sort	
• [] Appropriate and inappropriate use cases	
• [] Real-world applications	
• [] Conclusion and findings	

Introduction

Timsort is a hybrid, stable sorting algorithm, derived from merge sort and insertion sort, designed to perform well on many kinds of real-world data. It was implemented by Tim Peters in 2002 for use in the Python programming language. The algorithm finds subsequences of the data that are already ordered (runs) and uses them to sort the remainder more efficiently.

Methodology

Empirical Run-Time Testing Methodology

Data generation

The most obvious data set we should use is integers, however, that begs the question, what should be the range of the randomly generated integers?

Suppose that we generate random integers in the range between $[1, k]$ and we generate a list of size n . If $k \gg n$, then we would rarely get duplicates, meanwhile, if $k \ll n$, then we would get many duplicates.

Our parameter will then be the ratio $r = \frac{n}{k}$ where a high value of r would mean more duplicates (I think the duplicate can be thought of as the expected number of duplicates, although, I'm not too certain on the probabilistic analysis on that).

In our case, we selected 2 duplication ratios:

- $r = 10$ for data with high duplication
- $r = 0.1$ for data with low duplication

For each duplication ratios, we will use datasets of the following sizes:

- $n = 5$ for small datasets (for data with high duplication, $k = 1$, meaning that the data will just be an array containing just 1)
- $n = 1000$ for medium sized datasets
- $n = 1000000$ for large datasets

Computer Specifications

To make sure that the run time results aren't too much affected by other external factors, we ran the same algorithms on the same computer.

The most important variable for run time in this case is probably the CPU, and the speed and size of the RAM.

In our case, we used a CPU with the following `lscpu` output:

Architecture: x86_64

CPU op-mode(s):	32-bit, 64-bit
Address sizes:	39 bits physical, 48 bits virtual
Byte Order:	Little Endian
CPU(s):	6
On-line CPU(s) list:	0-5
Vendor ID:	GenuineIntel
Model name:	Intel(R) Core(TM) i5-9500 CPU @ 3.00GHz
CPU family:	6
Model:	158
Thread(s) per core:	1
Core(s) per socket:	6
Socket(s):	1
Stepping:	10
Microcode version:	0xfa
CPU(s) scaling MHz:	20%
CPU max MHz:	4400.0000
CPU min MHz:	800.0000
BogoMIPS:	6000.00
Flags:	fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc art ar ch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc cpuid aperfmperf pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 sdbg fma cx16 xtpr pdcm pcid sse4_1 s se4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave a vx f16c rdrand lahf_lm abm 3dnowprefetch cpuid_fault epb pti ssbd ibrs ibpb stibp tpr_shadow flexpriority ept vp id ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms i nvmcid mpx rdseed adx smap clflushopt intel_pt xsaveopt xsavec xgetbv1 xsaves dtherm ida arat pln pts hwp hwp_no tify hwp_act_window hwp_epp vnmi md_clear flush_l1d arch _capabilities
Virtualization features:	
Virtualization:	VT-x
Caches (sum of all):	
L1d:	192 KiB (6 instances)
L1i:	192 KiB (6 instances)
L2:	1.5 MiB (6 instances)
L3:	9 MiB (1 instance)
NUMA:	
NUMA node(s):	1
NUMA node0 CPU(s):	0-5
Vulnerabilities:	
Gather data sampling:	Mitigation; Microcode
Ghostwrite:	Not affected
Indirect target selection:	Not affected

Itlb multihit:	KVM: Mitigation: Split huge pages
L1tf:	Mitigation; PTE Inversion; VMX conditional cache flushes , SMT disabled
Mds:	Mitigation; Clear CPU buffers; SMT disabled
Meltdown:	Mitigation; PTI
Mmio stale data:	Mitigation; Clear CPU buffers; SMT disabled
Old microcode:	Not affected
Reg file data sampling:	Not affected
Retbleed:	Mitigation; IBRS
Spec rstack overflow:	Not affected
Spec store bypass:	Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1:	Mitigation; usercopy/swaps barriers and __user pointer sanitization
Spectre v2:	Mitigation; IBRS; IBPB conditional; STIBP disabled; RSB filling; PBRSE-eIBRS Not affected; BHI Not affected
Srbds:	Mitigation; Microcode
Tsa:	Not affected
Tsx async abort:	Mitigation; TSX disabled
Vmscape:	Mitigation; IBPB before exit to userspace

As for the RAM, the output of `sudo dmidecode --type 17` is as follows:

Getting SMBIOS data from sysfs.
SMBIOS 3.1.1 present.

Handle 0x000A, DMI type 17, 40 bytes
Memory Device

```

Array Handle: 0x0009
Error Information Handle: Not Provided
Total Width: 64 bits
Data Width: 64 bits
Size: 8 GiB
Form Factor: DIMM
Set: None
Locator: DIMM1
Bank Locator: Not Specified
Type: DDR4
Type Detail: Synchronous
Speed: 2666 MT/s
Manufacturer: 80CE000080CE
Serial Number: 343C8B13
Asset Tag: 02193900
Part Number: M378A1K43DB2-CTD
Rank: 1
Configured Memory Speed: 2666 MT/s
Minimum Voltage: Unknown
Maximum Voltage: Unknown

```

Configured Voltage: 1.2 V

Handle 0x000B, DMI type 17, 40 bytes

Memory Device

Array Handle: 0x0009

Error Information Handle: Not Provided

Total Width: Unknown

Data Width: Unknown

Size: No Module Installed

Form Factor: Unknown

Set: None

Locator: DIMM2

Bank Locator: Not Specified

Type: Unknown

Type Detail: None

The OS might also be a factor. The output of `uname -a` is as follows:

```
Linux archlinux 7.2.3-arch1-3 #1 SMP PREEMPT_DYNAMIC Sun, 06 Sep 2026 13:01:04 +0000 x86_64
```

The test would be decently fair when tested one after another because the CPU and memory utilization for other processes would be similar.

Data Collection Methods

Because of the randomness of the dataset, and the randomness of OS scheduling, it is necessary to run the code multiple time and record the average of all notable metrics alongside with its standard deviation.

Time complexity analysis

Performance results

Runtime

Conclusion